

Testing e oracolo

L'attività di testing del software si fonda su una condizione preliminare imprescindibile: è necessario conoscere a priori quale sia il comportamento aspettato dal sistema, in modo da poterlo confrontare con il comportamento effettivamente osservato durante il suo utilizzo.

Questa conoscenza fondamentale è detenuta da un'entità specifica chiamata "oracolo". L'oracolo può essere di natura umana, tipico dei test manuali, dove ci si basa su specifiche scritte o sulla conoscenza tacita del dominio.

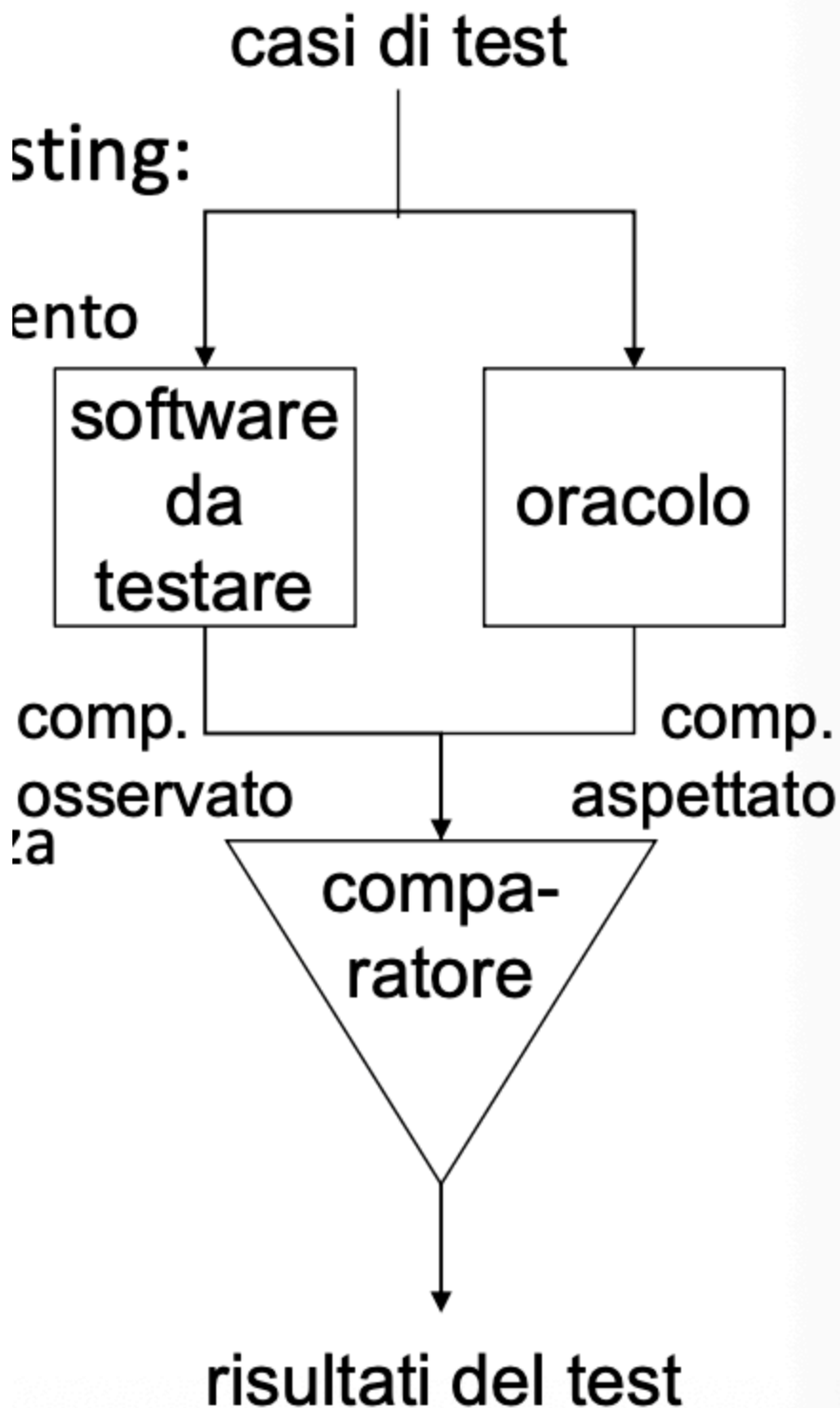
In alternativa, l'oracolo può essere automatico. In questo secondo scenario, il confronto avviene tramite specifiche formali (nel caso di test automatici), utilizzando lo stesso software sviluppato però da un team differente, oppure confrontando i risultati con una versione precedente del software stesso, pratica che prende il nome di test di regressione.

Per definire meglio l'attività, un test consiste nell'esecuzione di un programma guidata da uno o più casi di test.

L'esito di un test si considera di successo se la sua esecuzione non provoca alcun malfunzionamento, mentre si definisce fallito nel momento in cui genera un errore. L'elemento base di questo processo, ovvero il caso di test, è una descrizione dettagliata delle attività necessarie per effettuare il confronto tra il comportamento atteso e quello osservato.

Ogni caso di test include un sottoinsieme specifico di dati di ingresso e il relativo risultato (output) aspettato. L'insieme di tutti questi casi di test va a formare la cosiddetta "test

suite".



Problema della selezione dei casi di test

La teoria del testing si scontra spesso con i limiti pratici dell'informatica.

Teoricamente, un programma è considerato corretto solo se produce il risultato giusto per ogni possibile dato di ingresso.

Di conseguenza, un "test ideale" è quello in cui il successo del test garantisce in modo assoluto la correttezza dell'intero programma.

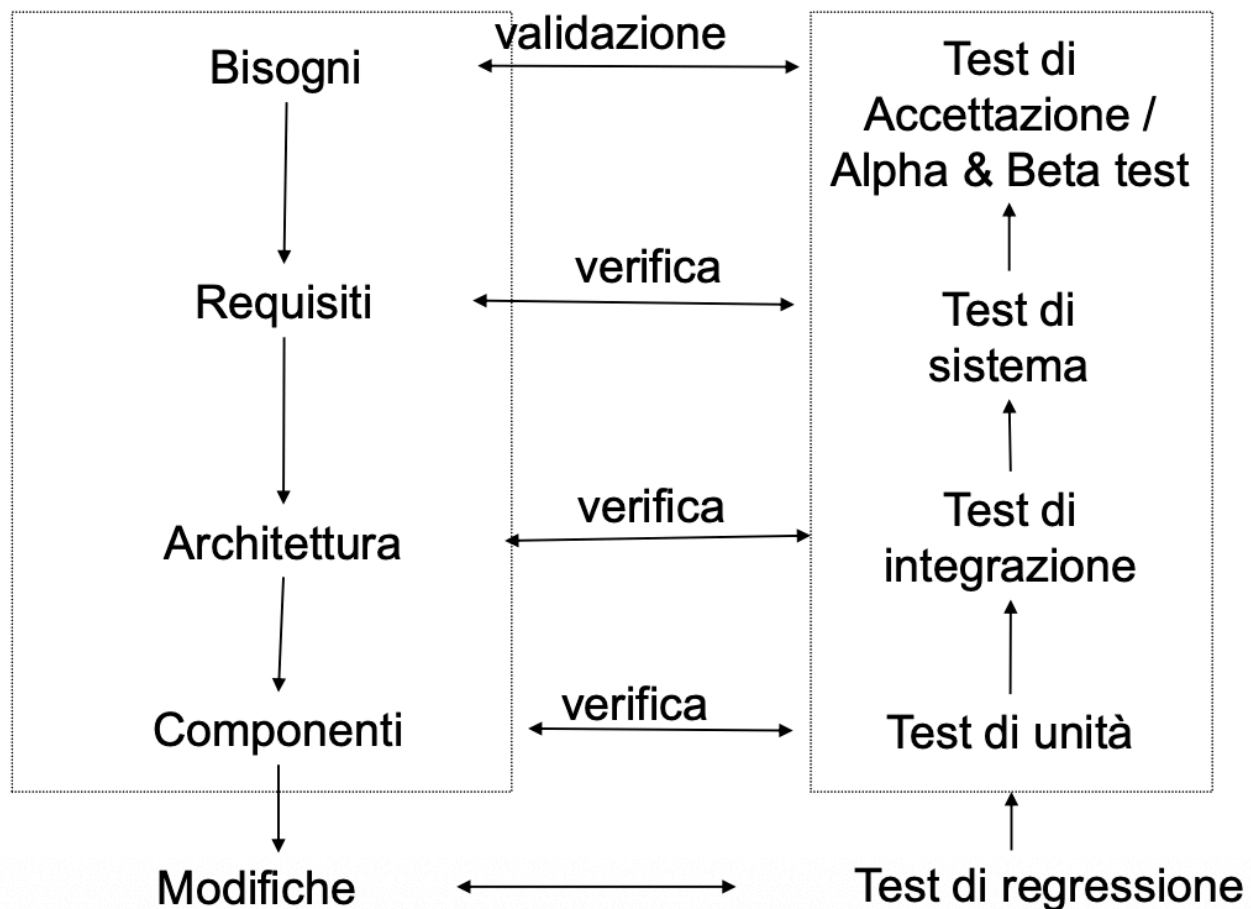
Questo richiederebbe un test di tipo esaustivo, capace di contemplare la totalità dei dati di ingresso. Sebbene un test esaustivo coincida con il test ideale , nella realtà lavorativa esso non è pratico e, quasi in ogni circostanza, risulta del tutto inattuabile.

A tal proposito, è celebre la tesi formulata da Dijkstra, secondo la quale l'attività di testing può essere utilizzata efficacemente per dimostrare la presenza di errori all'interno di un software, ma non potrà mai certificarne l'assoluta assenza. L'obiettivo realistico di chi sviluppa e testa il software diventa quindi quello di selezionare un gruppo di casi di test che possa approssimare il più possibile l'ideale. Si tratta di trovare un ragionevole compromesso ingegneristico: bilanciare il numero di malfunzionamenti che si riescono a scoprire con il numero complessivo di casi di test impiegati, ovvero la dimensione totale della test suite.

Terminazione

Un'altra questione cruciale è stabilire quando un programma possa considerarsi "testato a sufficienza". Non potendo eseguire test infiniti, sono stati codificati diversi criteri per decretare la fine delle attività:

- **Criterio temporale:** Il testing si conclude allo scadere di un periodo di tempo che è stato definito in precedenza.
- **Criterio di costo:** Le attività terminano quando viene esaurito lo sforzo (o il budget) precedentemente allocato per questa fase.
- **Criterio di copertura:** Ci si ferma al raggiungimento di una percentuale predefinita relativa agli elementi di un modello del programma. Un esempio classico è imporsi il raggiungimento del 100% di copertura delle linee di codice scritte.
- **Criterio statistico:** Si utilizza un valore MTBF (Mean Time Between Failure, ovvero il tempo medio tra i guasti) predefinito, il quale viene poi confrontato con un modello di affidabilità già esistente per valutare la maturità del sistema.



Mostra come diverse fasi di sviluppo (es. la definizione dei bisogni, dei requisiti o dell'architettura) corrispondano a diversi livelli di test (rispettivamente validazione e test di accettazione, verifica e test di sistema, verifica e test di integrazione).

I Test

In questa fase, il sistema viene collaudato direttamente dal committente all'interno del proprio ambiente di esecuzione.

Si possono verificare due scenari: se le caratteristiche del sistema risultano conformi alle specifiche concordate, il sistema viene formalmente accettato, qualora invece si rilevi una deviazione dalle specifiche, viene stilato un elenco dettagliato dei problemi e si avvia una fase di contrattazione con il committente per la loro risoluzione.

Quando il software sviluppato non è destinato a un singolo committente, ma è un prodotto rivolto a un vasto pubblico, si ricorre all'Alpha e al Beta test.

L'Alpha test coinvolge una rosa ristretta di utenti che utilizza il software al fine di segnalare eventuali problemi, concentrandosi principalmente sui requisiti funzionali.

Successivamente, il Beta test allarga la platea degli utenti coinvolti, spostando il focus dell'analisi sui requisiti non funzionali del prodotto.

A un livello più tecnico troviamo il test di sistema, in cui il software viene collaudato nella sua interezza, basandosi strettamente sulle specifiche dei requisiti originali.

Questa fase viene spesso affidata a personale specializzato e si divide in test funzionali e test mirati a verificare le performance, l'usabilità e la sicurezza (che rientrano nei requisiti non funzionali).

Test di integrazione e unità

Scendendo ulteriormente nel dettaglio opera il test di integrazione.

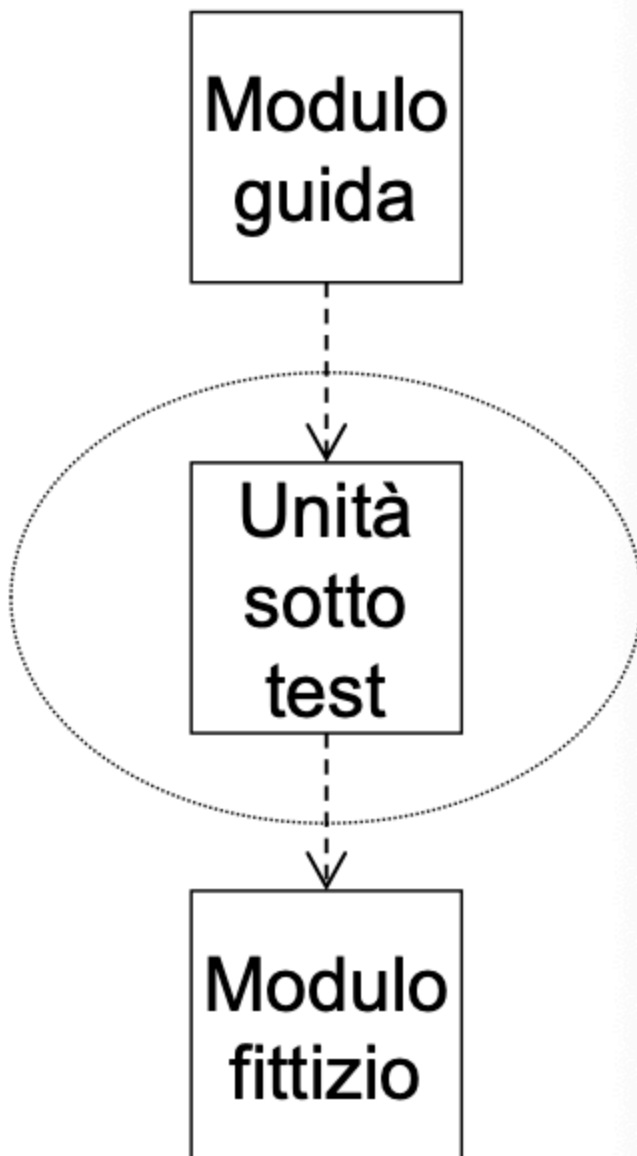
Il suo scopo principale è verificare che unità software differenti, pur essendo state sviluppate e testate in modo indipendente, riescano a lavorare insieme in modo appropriato.

L'obiettivo è quindi scovare i malfunzionamenti generati proprio dall'interazione tra queste diverse componenti.

Per farlo, si possono adottare due approcci principali: l'approccio "Big-bang", che prevede prima il collaudo delle singole unità e poi del sistema tutto insieme, oppure l'approccio incrementale, in cui le unità vengono progressivamente integrate nel sistema e testate passo dopo passo.

Infine il test di unità, le unità sono il frutto del lavoro individuale e vengono analizzate in totale isolamento dal resto del sistema, come avviene per le singole classi nella programmazione orientata agli oggetti.

Poiché l'unità è isolata, per poterla testare è necessaria la costruzione di componenti software ausiliari: i moduli "guida" (o *driver*), che hanno il compito di attivare l'unità sotto test, e i moduli "fittizi" (o *stub*), che simulano le parti di sistema di cui l'unità ha bisogno per funzionare. Oggi esiste un'ampia disponibilità di strumenti per automatizzare questi test di unità, come ad esempio il framework JUnit per i programmi scritti in linguaggio Java.



Criteri di selezione: Black-box e White-box

Per selezionare i casi di test a livello di unità, si utilizzano principalmente due famiglie di criteri, che sono spesso impiegate in modo complementare tra loro.

Da una parte abbiamo i criteri "**Black-box**", detti anche funzionali, i quali si basano esclusivamente sulle specifiche del software. L'idea è quella di testare il programma trattandolo come una scatola chiusa di cui non si conosce il codice sorgente. Tra questi criteri troviamo la suddivisione in classi di equivalenza, l'analisi dei valori limite e lo state-based testing.

Dall'altra parte troviamo i criteri "**White-box**", ovvero strutturali.

In questo caso, i casi di test sono selezionati proprio conoscendo la struttura interna del software.

Esempi di questo approccio includono la percentuale di copertura basata sulle istruzioni, l'analisi basata sul flusso di controllo o sul flusso dei dati, i criteri basati sul binding (il cosiddetto polymorphism testing) e il mutation testing.

Approfondimento sulle Tecniche Black-box

Entrando nel vivo delle tecniche funzionali, la "suddivisione in classi di equivalenza" mira a partizionare il dominio dei dati di ingresso.

L'assunto fondamentale è che se il programma risulta corretto per un singolo caso di test, si può dedurre ragionevolmente che sia corretto per ogni altro caso di test appartenente a quella stessa classe.

Queste classi di equivalenza rappresentano insiemi di stati validi o non validi per una specifica condizione di ingresso.

Una condizione può tradursi, ad esempio, in un intervallo di valori, in un valore specifico, in un elemento di un insieme discreto oppure in un semplice valore booleano.

Per procedere operativamente, la scomposizione del dominio segue regole che variano in base alla natura della condizione d'ingresso impostata nelle specifiche.

Quando la condizione è definita da un **intervallo di valori** (compreso tra un minimo e un massimo), il dominio viene frammentato in tre distinte classi: una classe valida, che accoglie tutti i valori interni all'intervallo, e due classi non valide, destinate rispettivamente ai valori inferiori al minimo e a quelli superiori al massimo.

Un comportamento analogo si applica quando la specifica richiede un **valore specifico** esatto, anche in questo scenario si generano tre classi, identificando una classe valida per il valore puntuale richiesto, una non valida per i valori inferiori e una non valida per quelli superiori.

La situazione si semplifica quando l'input deve far parte di un **insieme discreto** di elementi. In questo caso, l'algoritmo di scomposizione prevede la creazione di una sola classe valida, che raggruppa tutti gli elementi che appartengono legalmente all'insieme, e una classe non valida, che intercetta qualunque elemento esterno ad esso. Infine, nel caso in che la condizione d'ingresso sia un semplice **valore booleano**, la scomposizione mappa il dominio in due sole classi: una classe valida per gestire il valore logico TRUE e una classe non valida dedicata al valore logico FALSE.

La regola operativa prevede che ogni classe di equivalenza debba essere coperta da almeno un caso di test. Nello specifico, si seleziona un caso di test per ogni classe non valida e un caso di test per ogni classe valida, il quale può eventualmente essere condiviso da più classi valide contemporaneamente.

Per comprendere l'applicazione pratica di queste regole, si può fare riferimento all'esempio di una funzione progettata per il calcolo del fattoriale di un numero.

L'applicazione dei criteri di copertura delle classi di equivalenza porta alla modellazione di due casi di test iniziali:

- **Classe di equivalenza valida (numeri interi positivi):** Si definisce il caso di test **TC1**, in cui si imposta come dato di ingresso $n=4$ e ci si aspetta di osservare in output il valore corretto $n!=24$.
- **Classe di equivalenza non valida (numeri interi negativi):** Si definisce il caso di test **TC2**, in cui inserendo come dato di ingresso $n=-4$ ci si aspetta che il sistema rilevi l'anomalia restituendo come output la segnalazione di "input non valido".

Strettamente legata a questo concetto è "**l'analisi dei valori limite**".

L'esperienza insegna che i casi di test che esplorano le condizioni limite sono quelli che spesso rilevano la presenza di malfunzionamenti. Si vanno quindi a testare i valori che si trovano direttamente agli estremi, immediatamente al di sopra e immediatamente al di sotto delle classi di equivalenza d'ingresso o di uscita.

Ad esempio, per una funzione che calcola il fattoriale di un numero, oltre a testare interi positivi (classe valida) e interi negativi (classe non valida), è fondamentale verificare valori limite come 1, 0 o -1.

Riprendendo la medesima funzione per il calcolo del fattoriale, l'integrazione dell'analisi dei valori limite permette di irrobustire la test suite andando a sollecitare il codice proprio sui punti di frontiera delle classi d'ingresso e d'uscita. Questo approccio si concretizza nella formalizzazione di tre ulteriori casi di test cruciali:

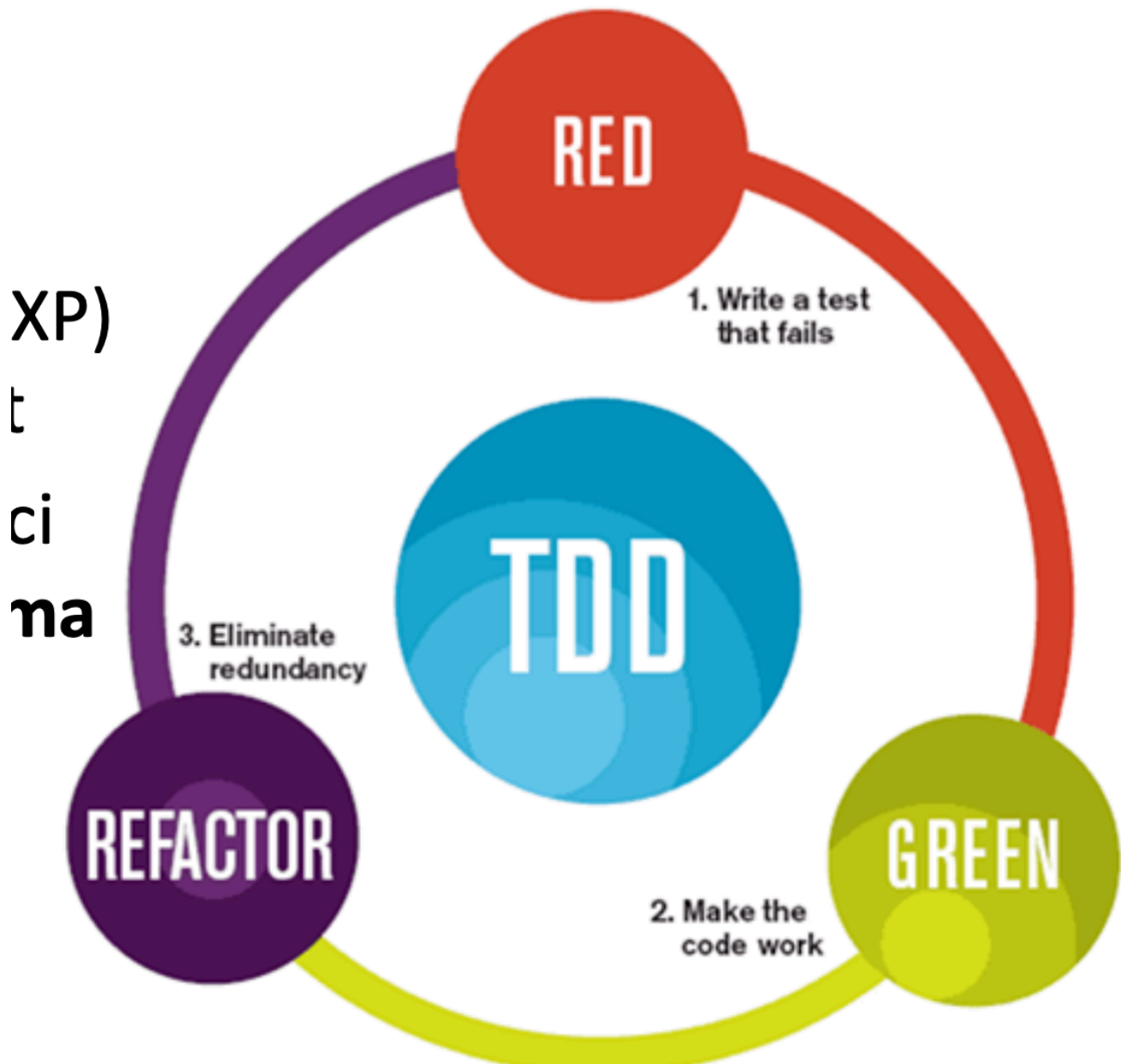
- **TC3 (Valore sul limite dell'intervallo valido):** Si testa il comportamento inserendo l'input minimale $n=1$, con l'aspettativa di ottenere come output $n!=1$.
- **TC4 (Valore di frontiera esatto):** Si esplora lo zero impostando l'input $n=0$, verificando che il sistema restituisca correttamente l'output $n!=1$ in accordo con la definizione matematica.
- **TC5 (Valore immediatamente al di sotto del limite valido):** Si sollecita la frontiera negativa impostando l'input $n=-1$, per accertarsi che il programma intercetti la condizione di errore e restituisca l'output di "input non valido".CACA

Il Test Driven Development (TDD)

Infine, il testo introduce il Test Driven Development (TDD). Si tratta di una pratica che deriva dall'Extreme Programming (XP) e che è strettamente legata allo Unit Test.

La particolarità rivoluzionaria del TDD risiede nel fatto che i test automatici vengono scritti prima ancora di scrivere il codice del programma. Questo approccio si fonda su un mantra molto preciso: "Red, Green, Refactor". Il ciclo di sviluppo si struttura in tre passaggi:

1. **Red:** Si scrive un test che, inevitabilmente, fallisce poiché la funzionalità non esiste ancora.
2. **Green:** Si implementa il codice strettamente necessario per far sì che il test abbia successo e il programma funzioni.
3. **Refactor:** Si ottimizza il lavoro fatto eliminando la ridondanza e migliorando la struttura del codice.



V&V: Verifica e Validazione

Nel contesto dell'ingegneria del software, l'acronimo V&V identifica due classi di attività distinte ma complementari, volte a garantire la qualità di un prodotto tecnologico/software. I due termini rispondono a quesiti profondamente diversi:

- **La Verifica:** Rappresenta l'insieme di tutte le attività ingegneristiche volte ad assicurare che il software realizzi correttamente una determinata funzione. L'obiettivo principale è accertarsi che il codice sia conforme alle specifiche tecniche definite nelle fasi precedenti dello sviluppo. In termini molto intuitivi, la verifica risponde alla domanda:
«*Stiamo costruendo il prodotto nel modo giusto?*».
- **La Validazione:** Si concentra invece sul piano macroscopico e sul rapporto con l'utente finale. Questa classe di attività serve a garantire che il software sia pienamente riconducibile ai bisogni effettivi e alle aspettative del cliente. La domanda filosofica e operativa a cui risponde la validazione è quindi:
«*Stiamo costruendo il prodotto giusto?*».

Attività di V&V

Per implementare concretamente le strategie di Verifica e Validazione, l'ingegnere del software può fare affidamento su due macro-categorie di attività, differenziate dal fatto che il codice venga eseguito o meno:

1. **Attività Statiche:** Sono analisi che non richiedono l'esecuzione del programma e si basano sull'esame logico e strutturale degli artefatti. Tra queste troviamo le *prove di correttezza*, governate da rigide specifiche formali e costrutti matematici. Troviamo poi l' *analisi statica del codice*, automatizzata tramite l'ausilio di tool specialistici (quali Checkstyle, Spotbugs, PMD o SonarQube) che scansionano i file sorgenti alla ricerca di violazioni sintattiche o vulnerabilità strutturali. Infine, rientrano in questo ambito le *review*, processi collaborativi basati sulla lettura e sulla discussione umana del codice, come avviene comunemente nelle moderne pull request.
2. **Attività Dinamiche:** Al contrario delle precedenti, queste attività si fondano sull'esecuzione diretta del software su una macchina. L'esempio cardine è l'attività di *testing*, la quale sollecita il programma tramite determinati vettori di input al fine esclusivo di far emergere i suoi malfunzionamenti. Un esempio classico di questa categoria è lo unit testing automatizzato tramite il framework JUnit in ambiente Java.

Malfunzionamenti vs Difetti

Un altro tassello cruciale per comprendere la teoria di V&V è la netta separazione concettuale tra ciò che si manifesta all'utente e ciò che risiede nascosto nel codice:

- **Malfunzionamento (Failure):** Rappresenta l'incapacità palese del software di comportarsi secondo le aspettative dell'utente o in conformità alle specifiche tecniche. Il malfunzionamento possiede per sua natura una *dimensione dinamica*: esso si manifesta in un preciso istante temporale e può essere osservato unicamente durante l'esecuzione effettiva del programma.

- **Difetto (Fault o Bug):** Rappresenta la manifestazione statica di un errore umano commesso durante la scrittura del codice o della progettazione. Il difetto è la causa profonda che, se stimolata dalle giuste condizioni d'uso, genera il malfunzionamento del sistema. A differenza della failure, il difetto ha una *natura statica*: esso esiste e rimane localizzato all'interno delle righe di codice indipendentemente dal fatto che il software venga eseguito o meno.